DS-GA 1011: Fundamentals of Natural Language Processing

Lecture 7: Post-training LLMs



Greg Durrett

Administrative details and recap

Supervised Fine-Tuning

SFT Data

RLHF

DPO

RLVR Intro

Basic RLVR: Expert Iteration,
Policy Gradient

GRPO

Administrative details and recap

Supervised Fine-Tuning

SFT Data

RLHF

DPO

RLVR Intro

Basic RLVR: Expert Iteration,
Policy Gradient

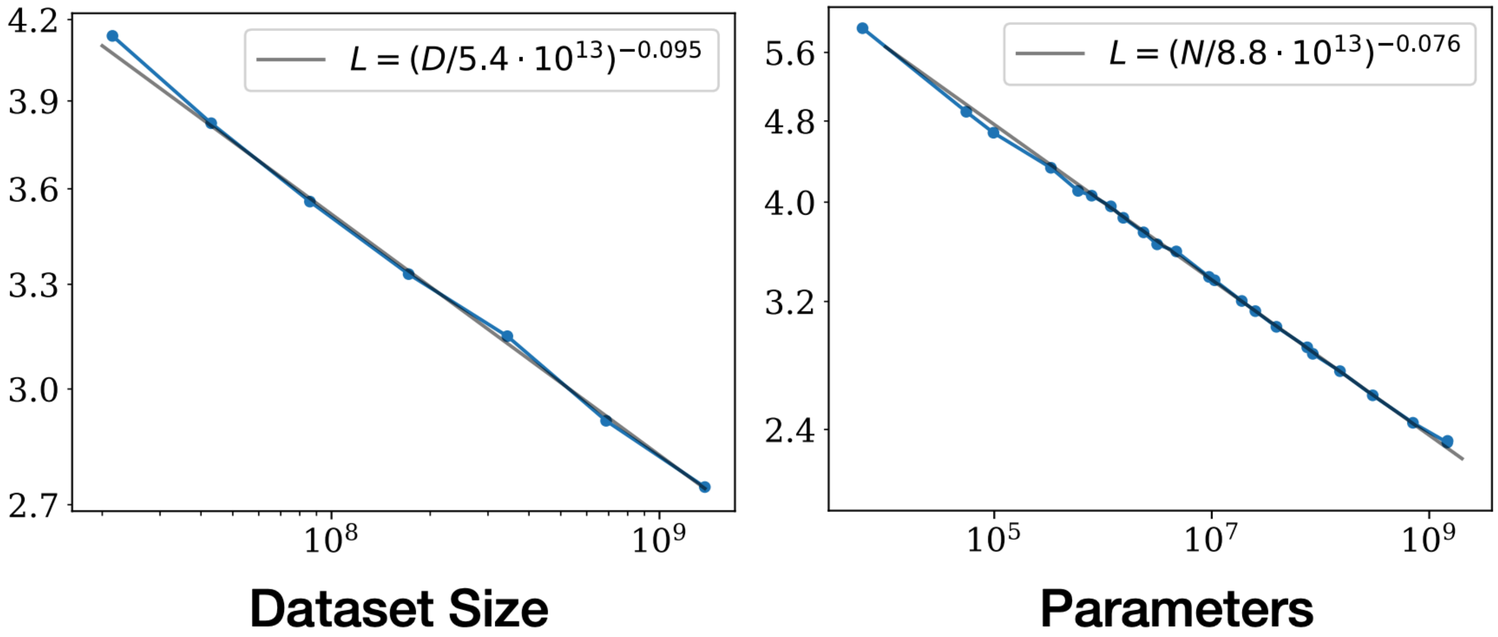
GRPO

Administrative details and recap

Administrivia

- Midterm next week
  - Note sheet allowed
  - Comprehensive exam of the course so far, including some longer-form problems
  - Includes today's lecture!
- Assignment 3 due next week, Assignment 4 released next week
- Final project spec released after midterm, proposals due the following week

Scaling Laws

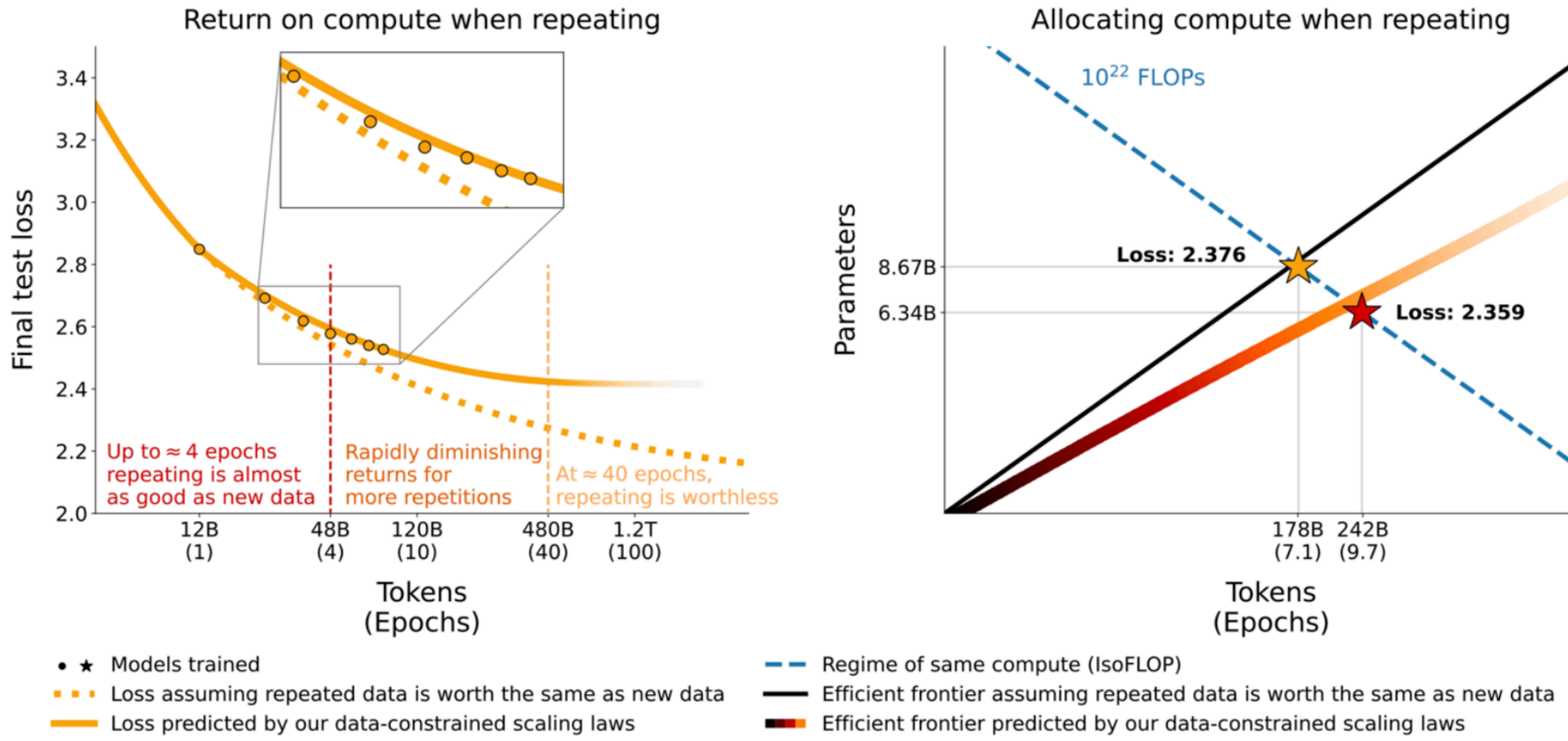


- What could scaling laws do for us?

Kaplan et al. (2020)

Q1: When to train on repeated data?

In practice, we have finite data – how does repeating examples affect scaling?



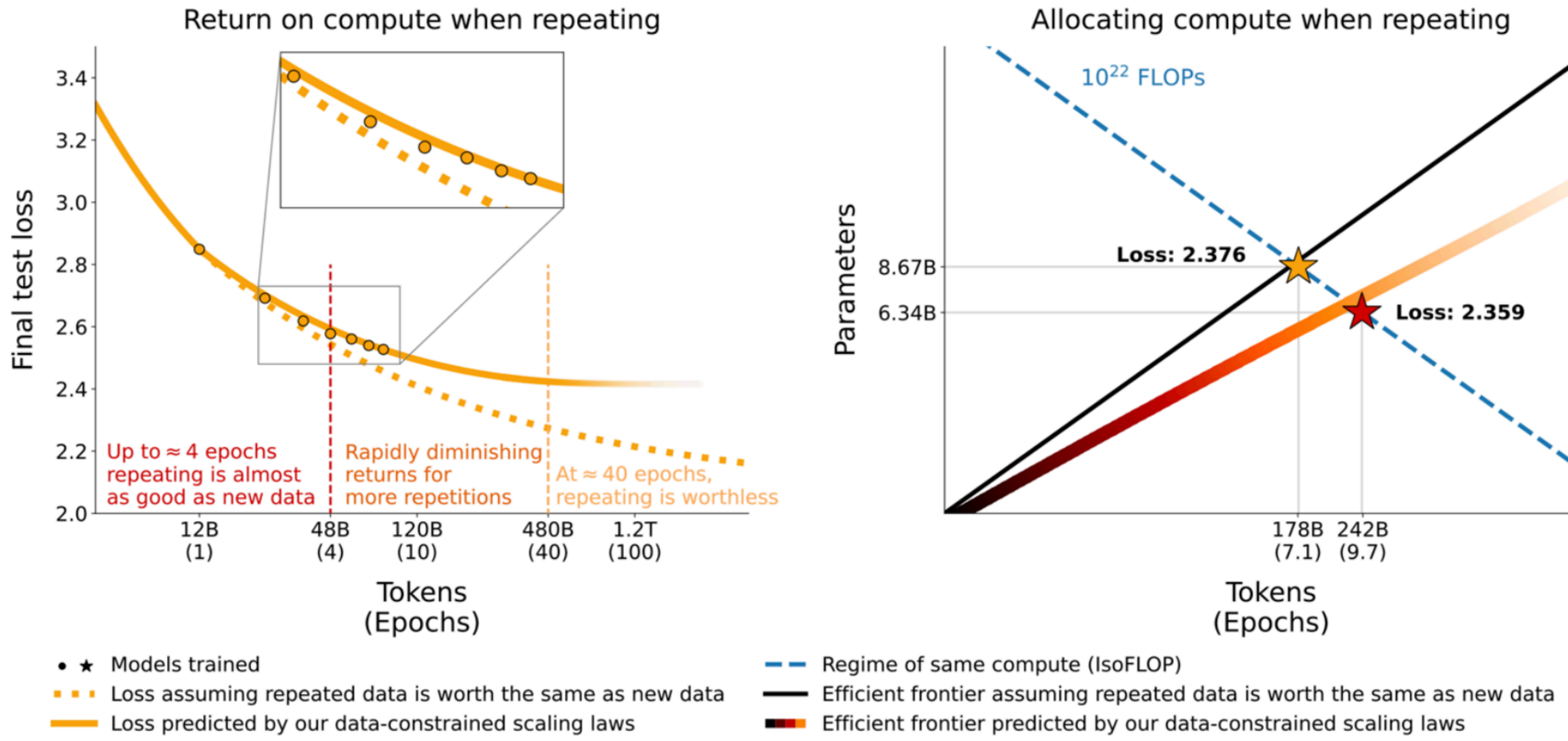
Muennighoff et al. (2023)

$$D' = U_D + U_D R_D^* (1 - e^{\frac{-R_D}{R_D^*}}) \ .$$

D' = Effective data
Ud = Unique tokens
Rd* = Constant
Rd = Repetition

Q1: When to train on repeated data?

In practice, we have finite data – how does repeating examples affect scaling?



Muennighoff et al. (2023)

$$D' = U_D + U_D R_D^* (1 - e^{\frac{-R_D}{R_D^*}}) .$$

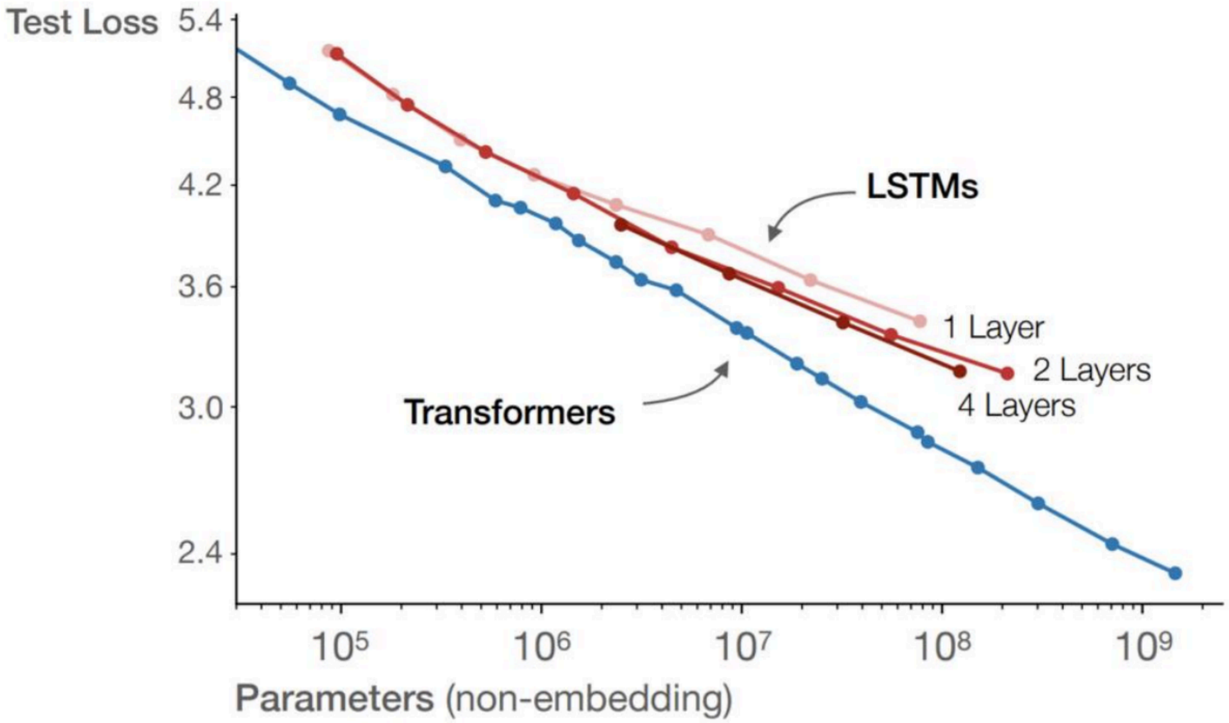
D' = Effective data
Ud = Unique tokens
Rd* = Constant
Rd = Repetition

Q2: What architecture to use?

Q: Are transformers better than LSTMs?

Brute force way: spend tens of millions to train a LSTM GPT-3

Scaling law way:



[Kaplan+ 2021]

Q3: How big a model to train?

Pick a range of FLOP budgets, vary the total parameter count, take the min over these convex shapes. The minima form a power law.

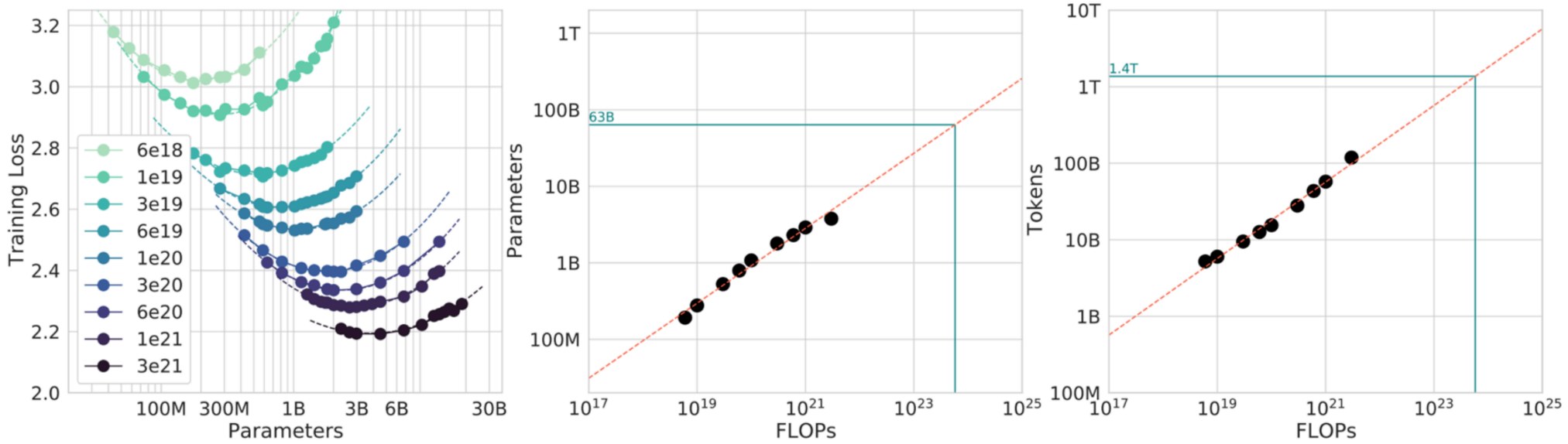
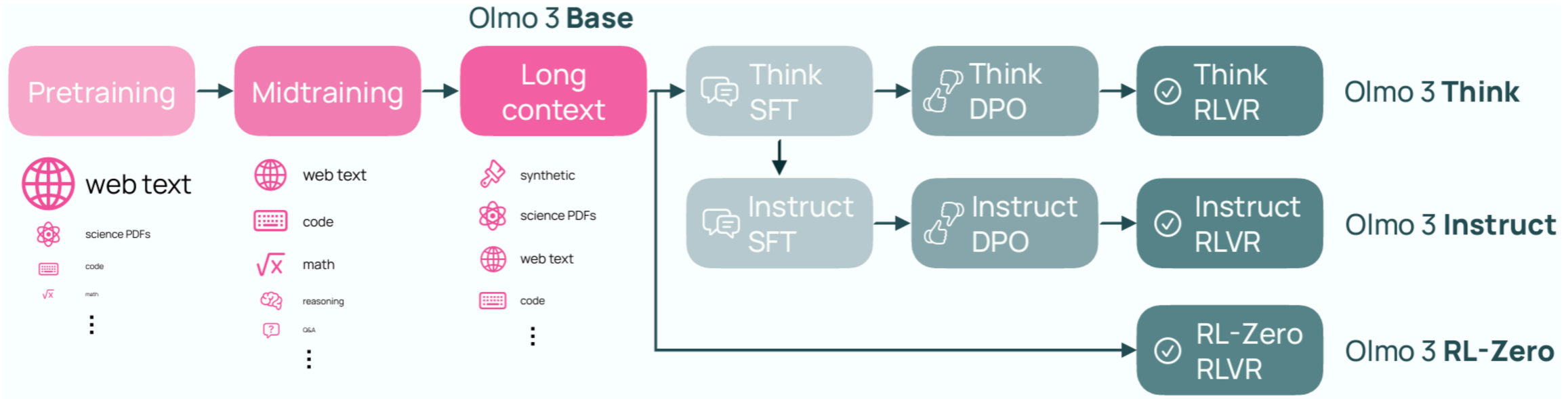


Figure 3 | **IsoFLOP curves.** For various model sizes, we choose the number of training tokens such that the final FLOPs is a constant. The cosine cycle length is set to match the target FLOP count. We find a clear valley in loss, meaning that for a given FLOP budget there is an optimal model to train (**left**). Using the location of these valleys, we project optimal model size and number of tokens for larger models (**center** and **right**). In green, we show the estimated number of parameters and tokens for an *optimal* model trained with the compute budget of *Gopher*.

Today: Post-training



Administrative details and recap

Supervised Fine-Tuning

SFT Data

RLHF

DPO

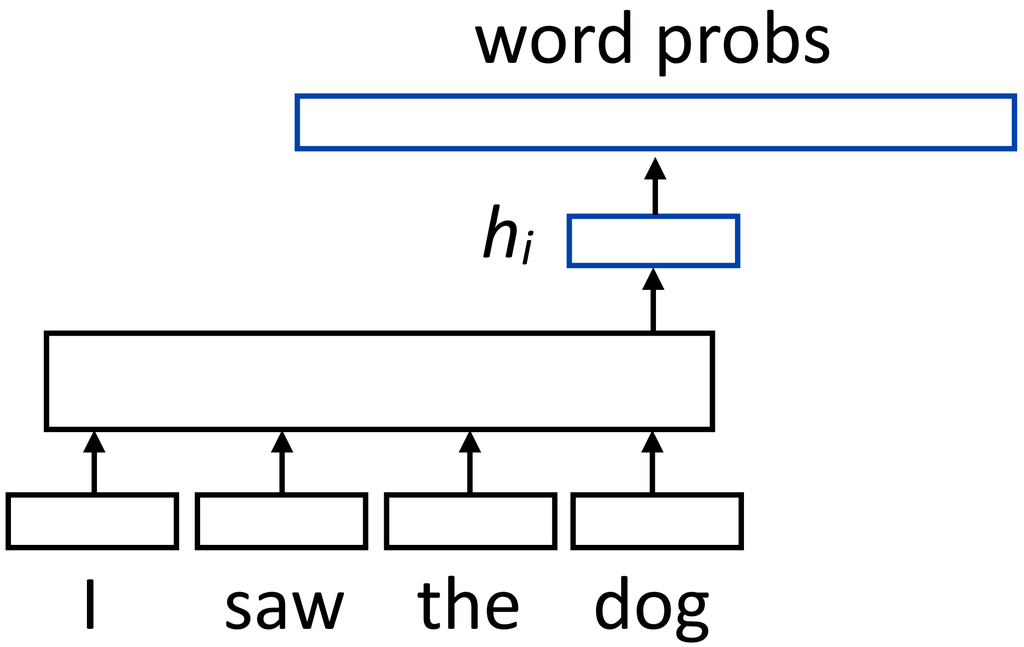
RLVR Intro

Basic RLVR: Expert Iteration,
Policy Gradient

GRPO

Supervised Fine-Tuning

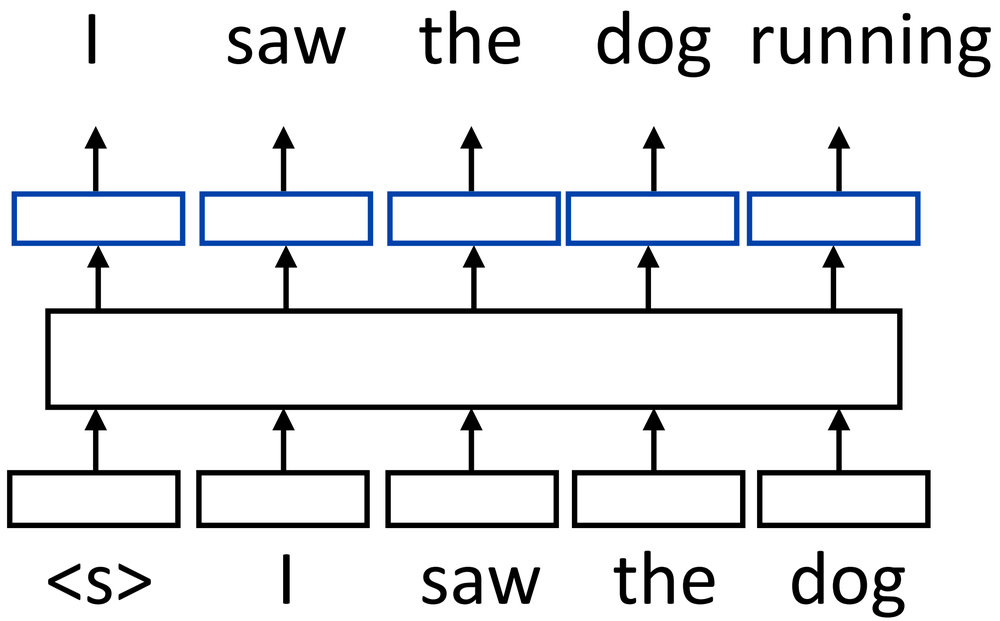
Transformer Language Modeling



$$P(w|\text{context}) = \text{softmax}(W\mathbf{h}_i)$$

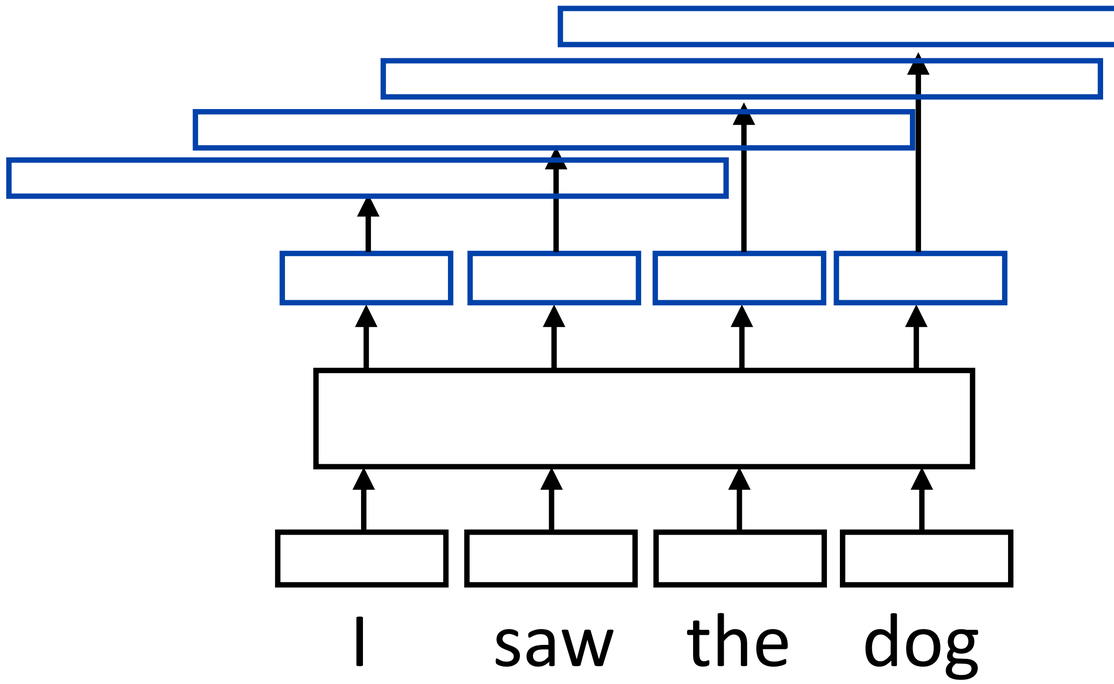
W is a (vocab size) x (hidden size) matrix

Training Transformer LMs



- Input is a sequence of words, output is those words shifted by one
- Allows us to train on predictions across several timesteps simultaneously

Training Transformer LMs



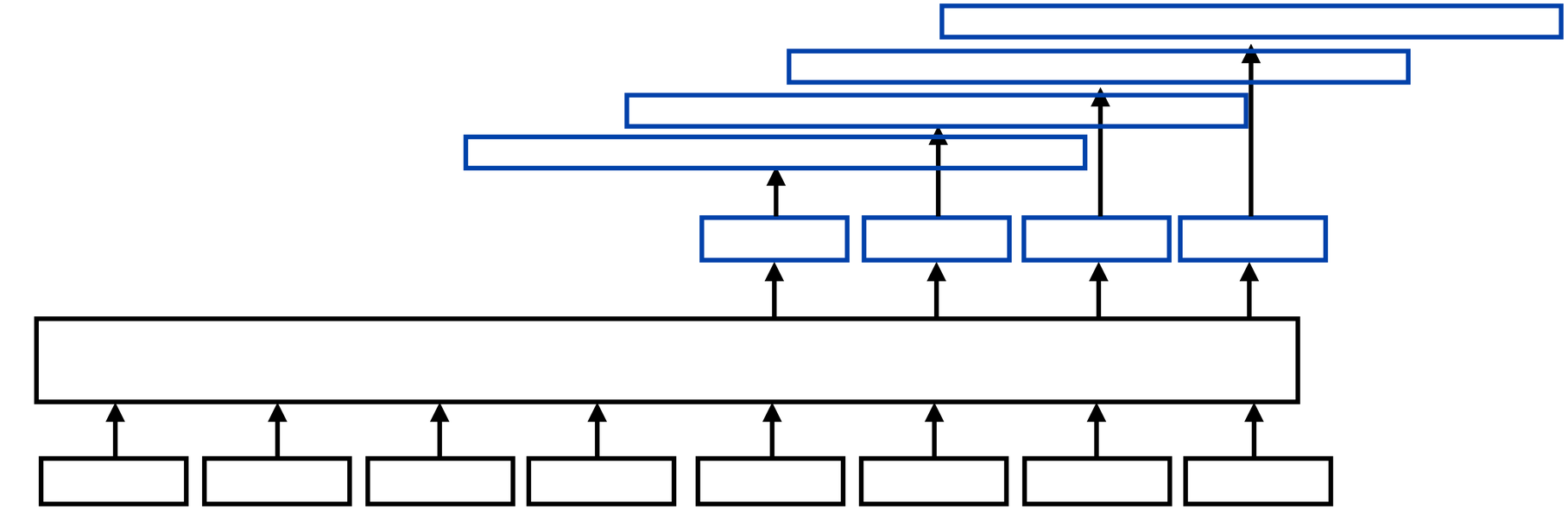
P(w|context)

loss = — log P(w*|context)

Total loss = sum of negative log likelihoods at each position

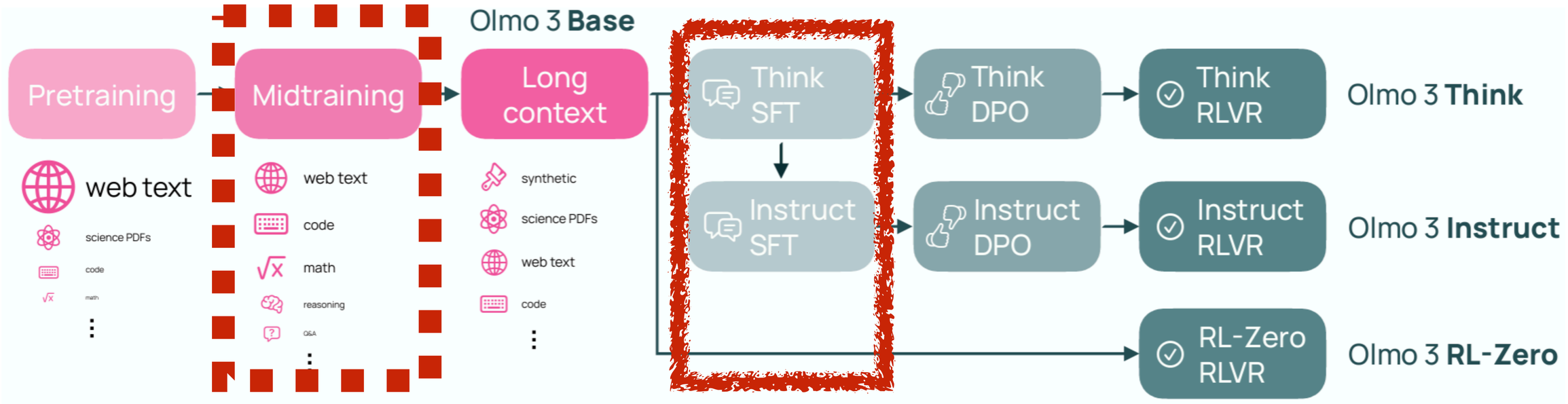
- Parallel inference across several tokens at training time, but at decoding time, tokens are generated one at a time

What is SFT?



- Same as language model training, but only compute loss on the response. Learn to respond to questions given prompts.

When is SFT?



Kind of SFT!
(similar data but train over all tokens, not just responses)

SFT!

Why is SFT?

- LLM pre-training can be considered either unsupervised or self-supervised
  - Unsupervised: we're just learning a function p(x) representing our data (the web)
  - Self-supervised: it's supervised prediction (predict the next word), but the labels are "free"

- This isn't the same as supervised learning, where we tell a model exactly what outputs we want it to return. **SFT lets us do this.**

- Where is pre-training on the web going to differ from what we want?

What was the "shot heard round the world"?

The shot that was heard round the world was fired in Lexington, MA. It was fired because, at the time, America was still under British rule. The British soldiers had been taking firewood by throwing it down from the top of their fort at Lexington. Someone who was not a soldier, by the name of Joseph Warren (who had ties to the Sons of Liberty or the Sons of Freedom) shouted to the British to stop doing it. The British soldiers ignored him, not

What do you think of this completion?

What was the "shot heard round the world"?

"What to the slave is the 4th of July?"

What was the sideline occupation of Thomas Jefferson's wife, Martha, and their enslaved girl, Sally Hemmings?

What was the name of the first African American regiment from Vermont?

What do you think of this one?

Samples from GPT-3

Why SFT? Safety

Truthful

Unbiased

Non-toxic

Harmless (e.g., don’t help users build a bomb)

More later in the semester!

Administrative details and recap

Supervised Fine-Tuning

SFT Data

RLHF

DPO

RLVR Intro

Basic RLVR: Expert Iteration,
Policy Gradient

GRPO

SFT Data

Slide credit: Tatsu Hashimoto

FLAN: Examples from published datasets

Stephanie - Can you finalize the attached and have it signed. I need to initial it, but it needs to be signed by Brad Richter. Thanks. Write a subject line for this email.	Ronald Chisholm LOI
Ahold to Sell Spain Operations to Permira (AP) AP - The Dutch supermarket retailer Ahold, seeking to streamline global operations and reduce debt, said Sunday it will sell its holdings in Spain to Permira Funds for about #36;849 million. What is this text about? OPTIONS: - World - Sports - Business - Science/Tech	Business
Write highlights for this article: Sauntering down leafy avenues past typical Dutch step-gabled buildings, carpets of crocuses, the occasional cyclist whooshing quietly by and the sun bouncing off the canal, I can think of few more pleasant places to spend a spring weekend than The Hague. […]The most prized Dutch colony was apparently run from The Hague (aka 'the Widow of Indonesia'). For summer: the excellent Grapes & Olives and the Café de Bieb on Veenkade will have boats on a newly renovated bit of canal, with music and boat trips planned, including a tour at the Kompaan brewery (I can vouch for the product). My art gorging finishes at the Panorama, an extraordinary, 14 ft-high circular panorama of The Hague's seaside resort, Scheveningen, painted by Hendrik Mesdag. I take a 15-minute tram to the place itself and gorge on scallops and sole at Catch by Simonis on the harbour front. The Hague has netted a new fan. British Airways (0844 493 0787, www.ba.com) flies to Rotterdam Den Haag (half-hour taxi) from £100 return. For tours with Remco Dörr, email remco.dorr@hotmail.com. Double rooms at the Carlton Ambassador (00 31 70 363 0363, www.carlton.nl/ambassador) cost from £84, room only. More information on travel in The Netherlands at www.holland.com.	The least known of the Dutch cities, The Hague was a village until 1806. It owes its growth to Louis Bonaparte, Napoleon's brother, who ruled here. The city has a wealth of art, including Vermeer's 'Girl With A Pearl Earring'
Here is some data about a restaurant: name = Aromi, eatType = coffee shop, food = English, customer rating = 5 out of 5, area = city centre. Write a sentence that includes the following data about a restaurant	In the city centre there is a coffee shop with a customer rating of 5 out of 5 called Aromi which serves English food.

Slide credit: Tatsu Hashimoto

Alpaca: Examples from GPT-3

Give three tips for staying healthy.	1.Eat a balanced diet and make sure to include plenty of fruits and vegetables. 2. Exercise regularly to keep your body active and strong. 3. Get enough sleep and maintain a consistent sleep schedule.
What does “algorithm” mean?	An algorithm is a step-by-step set of instructions used to solve a problem or achieve a goal. It is a process of breaking down complex tasks into smaller, more manageable steps. Algorithms can be used to find the shortest path from one place to another, sort data, or render data from an immense database in a few seconds.
Find the average number in a list.	def avg_list(nums): total = 0 for n in nums: total += n return total / len(nums) print(avg_list([4, 3, 6, 10, 8])) # Output 6.4

Slide credit: Tatsu Hashimoto

OpenAssistant

Can you write a short introduction about the relevance of the term "monopsony" in economics? Please use examples related to potential monopsonies in the labour market and cite relevant research.	"Monopsony" refers to a market structure where there is only one buyer for a particular good or service. In economics, this term is particularly relevant in the labor market, where a monopsony employer has significant power over the wages and working conditions of their employees. […]. Overall, the concept of monopsony is essential to understanding the dynamics of labor markets and the impact of market power on workers. Further research is needed to understand the extent and impact of monopsonies on the economy and to develop policies to address this issue. References: Bivens, J., & Mishel, L. (2013). The Pay of Corporate Executives and Financial Professionals as Evidence of Rents in Top 1 Percent Incomes. Journal of Economic Perspectives, 27(3), 57-78.
My kid needs to do a science project for elementary school, what are some cheap but fun projects to do?	There are many cheap and fun science projects that elementary school kids can do! Here are a few ideas: 1. Make a homemade lava lamp: Fill a clear plastic bottle with water and vegetable oil, add a few drops of food coloring, and then drop in an Alka-Seltzer tablet. The reaction will cause bubbles to rise and fall like a lava lamp. […]7. Create a solar oven: Using a pizza box, aluminum foil, and plastic wrap, create a solar oven that can cook s'mores or other small snacks. 8. Explore magnets: Using magnets and a variety of objects, have your child experiment with magnetism to see what objects are attracted to magnets and which ones are not. These are just a few ideas to get you started! There are many more cheap and fun science projects that your child can do with household items and a little creativity.

Slide credit: Tatsu Hashimoto

Response Lengths Vary

Table 1: Instruction datasets investigated in this work. CoT and FLAN V2 are sampled to 100K to match the sizes of other datasets. We report the average number of conservation turns ($\bar{N}_{rounds}$), average length of prompts ($\bar{L}_{prompt}$), average length of completion ($\bar{L}_{completion}$).

Datasets	**Sourced from**	**# Instances**	$\bar{N}_{rounds}$	$\bar{L}_{prompt}$	$\bar{L}_{completion}$
SuperNI [48]	NLP datasets + Human-written Instructions	96,913	1.0	291.1	38.7
CoT [50]	NLP datasets + Human-written CoTs	100,000	1.0	266.0	53.2
Flan V2 [31]	NLP datasets + Human-written Instructions	100,000	1.0	355.7	31.2
Dolly [12]	Human-written from scratch	15,011	1.0	118.1	91.3
Open Assistant 1 [26]	Human-written from scratch	34,795	1.6	34.8	212.5
Self-instruct [47]	Generated w/ vanilla GPT3 LM	82,439	1.0	41.5	29.3
Unnatural Instructions [23]	Generated w/ Davinci-002	68,478	1.0	107.8	23.6
Alpaca [43]	Generated w/ Davinci-003	52,002	1.0	27.8	64.6
Code-Alpaca [6]	Generated w/ Davinci-003	20,022	1.0	35.6	67.8
GPT4-Alpaca [36]	Generated w/ Davinci-003 + GPT4	52,002	1.0	28.0	161.8
Baize [52]	Generated w/ ChatGPT	210,311	3.1	17.6	52.8
ShareGPT[3]	User prompts + outputs from various models	168,864	3.2	71.0	357.8

Yizhong Wang et al. (2023) TULU

Impact of these Datasets

Table 3: Comparison of different instruction tuning datasets, showing that different instruction-tuning datasets can excel in different aspects, and mixtures perform best on average. Cells are blue if the finetuning boosts the vanilla LLAMA performance, and orange if the finetuning hurts the performance.

	MMLU (factuality)	GSM (reasoning)	BBH (reasoning)	TydiQA (multilinguality)	Codex-Eval (coding)	AlpacaEval (open-ended)	Average
	EM (0-shot)	EM (8-shot, CoT)	EM (3-shot, CoT)	F1 (1-shot, GP)	P@10 (0-shot)	Win % vs Davinci-003	
Vanilla LLaMa 13B	42.3	14.5	39.3	43.2	28.6	-	-
+SuperNI	49.7	4.0	4.5	**50.2**	12.9	4.2	20.9
+CoT	44.2	40.0	41.9	47.8	23.7	6.0	33.9
H +Flan V2	**50.6**	20.0	40.8	47.2	16.8	3.2	29.8
H +Dolly	45.6	18.0	28.4	46.5	31.0	13.7	30.5
H +Open Assistant 1	43.3	15.0	39.6	33.4	31.9	58.1	36.9
+Self-instruct	30.4	11.0	30.7	41.3	12.5	5.0	21.8
+Unnatural Instructions	46.4	8.0	33.7	40.9	23.9	8.4	26.9
+Alpaca	45.0	9.5	36.6	31.1	29.9	21.9	29.0
G +Code-Alpaca	42.5	13.5	35.6	38.9	34.2	15.8	30.1
G +GPT4-Alpaca	46.9	16.5	38.8	23.5	**36.6**	63.1	37.6
+Baize	43.7	10.0	38.7	33.6	28.7	21.9	29.4
G +ShareGPT	49.3	27.0	40.4	30.5	34.1	**70.5**	42.0
H +Human data mix.	50.2	38.5	39.6	47.0	25.0	35.0	39.2
H+G +Human+GPT data mix.	49.3	**40.5**	**43.3**	45.6	35.9	56.5	**45.2**

Yizhong Wang et al. (2023) TULU

Administrative details and recap

Supervised Fine-Tuning

SFT Data

RLHF

DPO

RLVR Intro

Basic RLVR: Expert Iteration,
Policy Gradient

GRPO

RLHF

RLHF: Introduction

- “Reinforcement learning from human feedback”
- In 2022, fine-tuning on labeled datasets was commonplace (T5/T0, Flan, other task-specific models from 2019-2022)
- These models could do many tasks, but didn’t feel broadly useful, and they weren’t “chatbots” as we think of them now
- Big shift with ChatGPT in November 2022: RLHF-tuned models

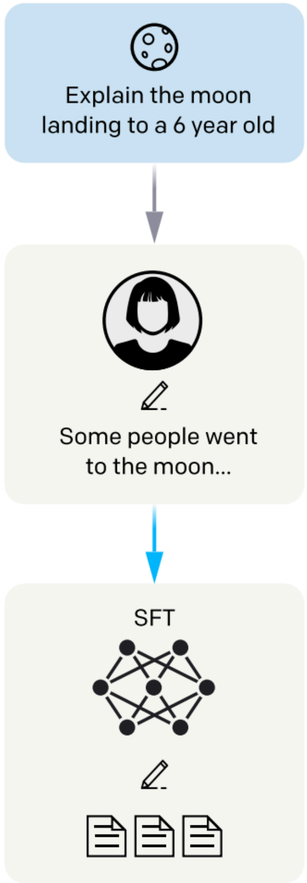
RLHF

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.

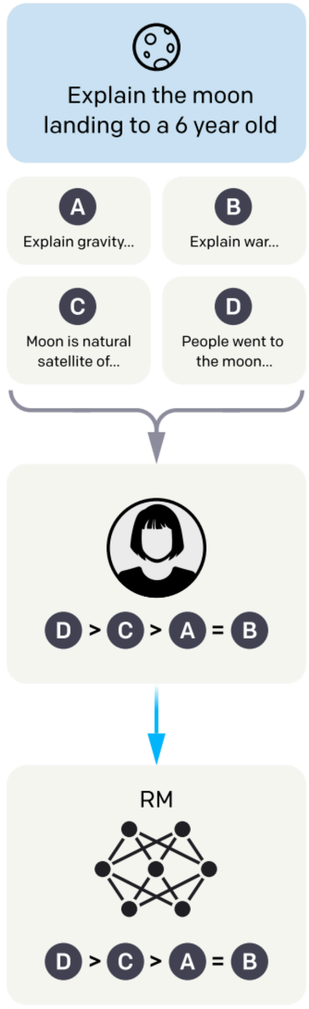


Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



- Apply this approach to optimizing outputs from large language models
- Step 3 (not shown): do RL with this policy

Ouyang et al. (2022)

Learning Reward Models

- Input **x**: *who was the US president during World War II?*
- Outputs $\mathbf{y}^+$: *Franklin D. Roosevelt, Harry Truman*

- Classical RL: assign some value +3 to this output

- Should we just get humans to label rewards? What scale do we use? What score should this get?

Ouyang et al. (2022)

Learning Reward Models

- Input **x**: *who was the US president during World War II?*
- Outputs $\mathbf{y}^+$: *Franklin D. Roosevelt, Harry Truman*

  $\mathbf{y}^-$: *Herbert Hoover, Franklin D. Roosevelt, Harry Truman*

$$P(y^+ \succ y^- \mid \mathbf{x}) = \frac{\exp(r(y^+, \mathbf{x}))}{\exp(r(y^+, \mathbf{x})) + \exp(r(y^-, \mathbf{x}))}$$

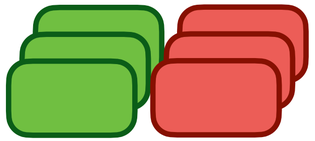
- Bradley-Terry model: turns scores into log probabilities of 1 being preferred to 2. Same as logistic regression where we classify pairs as 1 > 2 or 2 < 1, but we learn a continuous scoring function

Ouyang et al. (2022)

Learning Reward Models

- Input **x**: *who was the US president during World War II?*
- Outputs **y**$^{+}$: *Franklin D. Roosevelt, Harry Truman*

  y^{-}: *Herbert Hoover, Franklin D. Roosevelt, Harry Truman*



Lots of (**y**$^{+}$,**y**$^{-}$) pairs

$$\rightarrow \; P(y^+ \succ y^- \mid \mathbf{x}) = \frac{\exp(r(y^+, \mathbf{x}))}{\exp(r(y^+, \mathbf{x})) + \exp(r(y^-, \mathbf{x}))}$$

- Outcome: reward model *r*(y, **x**) returning real-valued scores

Ouyang et al. (2022)

RLHF

- Goal: find a policy π_θ (LM parameters) that optimizes the following:

$$R(\mathbf{x}, y) = r(\mathbf{x}, y) - \lambda D_{\mathrm{KL}}(\pi_\theta(y \mid \mathbf{x}) \| \pi_\theta^{\mathrm{SFT}}(y \mid \mathbf{x}))$$

get high reward

stay close to an initial SFT policy

- This is called *proximal policy optimization* (PPO)
- Important to regularize towards the SFT policy! Reward models are not stable enough to make things work
- PPO has some details in its implementation: it's an *advantage actor-critic* model, so there's a separate value function that gets learned

Christiano et al. (2017)

RLHF

Table 1: Distribution of use case categories from our API prompt dataset.

Use-case	(%)
Generation	45.6%
Open QA	12.4%
Brainstorming	11.2%
Chat	8.4%
Rewrite	6.6%
Summarization	4.2%
Classification	3.5%
Other	3.5%
Closed QA	2.6%
Extract	1.9%

Table 2: Illustrative prompts from our API prompt dataset. These are fictional examples inspired by real usage—see more examples in Appendix A.2.1.

Use-case	Prompt
Brainstorming	List five ideas for how to regain enthusiasm for my career
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.
Rewrite	This is the summary of a Broadway play: """ {summary} """ This is the outline of the commercial for that play: """

- For OpenAI, RLHF data is collected from their API. **Very different from instruct-tuning datasets**

Ouyang et al. (2022)

Administrative details and recap

Supervised Fine-Tuning

SFT Data

RLHF

DPO

RLVR Intro

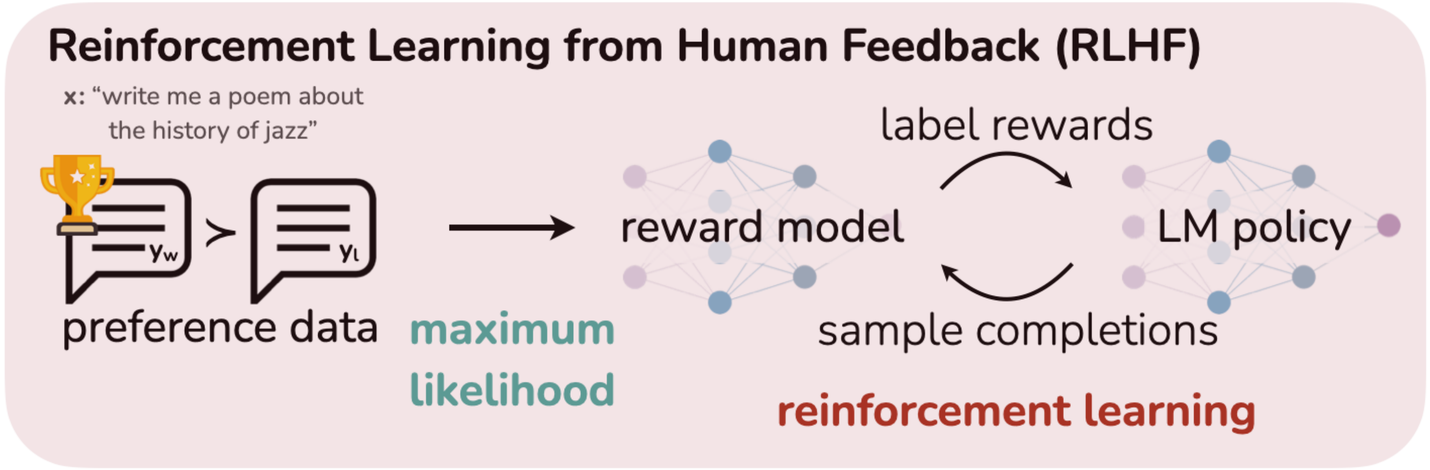
Basic RLVR: Expert Iteration,
Policy Gradient

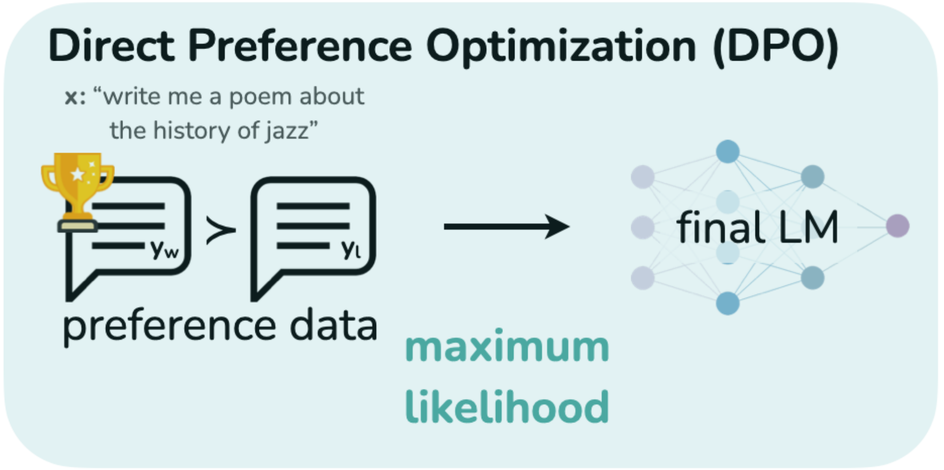
GRPO

DPO

Direct Preference Optimization (DPO)

- Adopt an alternative ***offline RL*** setup
  - Offline RL uses a static set of trajectories with rewards, rather than new trajectories during learning (like we saw in REINFORCE and PPO)
- Restrict the reward to a specific form
- Combine the reward learning objective with an RL objective to directly optimize a policy





Slide credit: Yoav Artzi

Direct Preference Optimization (DPO)

- DPO starts with a very similar RL objective to PPO

$$\arg\max_\theta E_{\bar{x}\sim\mathcal{D},\bar{y}\sim\pi_\theta(\bar{y}|\bar{x})}\left[r(\bar{x},\bar{y}) - \beta\mathrm{KL}[\pi_\theta(\bar{y}\,|\,\bar{x}),\pi_{\text{ref}}(\bar{y}\,|\,\bar{x})]\right]$$

  - Where π_{ref} is the SFT policy before we fine-tune it with preference data

Maximize the expected reward according to our prompt data and policy

Penalize for the distribution getting further from the pre-RL distribution

Slide credit: Yoav Artzi

Direct Preference Optimization (DPO)

- DPO starts with a very similar RL objective to PPO

$$\arg\max_\theta E_{\bar{x}\sim\mathcal{D},\bar{y}\sim\pi_\theta(\bar{y}|\bar{x})}\left[r(\bar{x},\bar{y}) - \beta\mathrm{KL}[\pi_\theta(\bar{y}\,|\,\bar{x}),\pi_{\text{ref}}(\bar{y}\,|\,\bar{x})]\right]$$

  - Where π_{ref} is the SFT policy before we fine-tune it with preference data

- The optimal policy takes this form (according to theoretical results from RL)

$$\pi^*(\bar{y}\,|\,\bar{x}) = \frac{1}{Z(\bar{x})}\pi_{\text{ref}}(\bar{y}\,|\,\bar{x})\exp\left(\frac{1}{\beta}r(\bar{x},\bar{y})\right)$$

- We can rearrange that to give:

$$r(\bar{x},\bar{y}) = \beta\log\frac{\pi^*(\bar{y}\,|\,\bar{x})}{\pi_{\text{ref}}(\bar{y}\,|\,\bar{x})} + \beta\log Z(\bar{x})$$

- Combine this with Bradley-Terry and…

Slide credit: Yoav Artzi

Direct Preference Optimization (DPO)

- Through some manipulation, it can be shown that the optimal policy π^* for RLHF satisfies the preference model

$$p^*(y_1 \succ y_2 \mid x) = \frac{1}{1 + \exp\left(\beta \log \frac{\pi^*(y_2|x)}{\pi_{\text{ref}}(y_2|x)} - \beta \log \frac{\pi^*(y_1|x)}{\pi_{\text{ref}}(y_1|x)}\right)}$$

ref = SFT policy. preferred output should be more likely under our learned policy than under reference, dispreferred output should be less likely

- We can now learn the policy directly to optimize the log likelihood of the preference data in a fashion that looks like supervised learning:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

Rafailov et al. (2023)

Direct Preference Optimization (DPO)

- The DPO gradient is:

$$\nabla \mathcal{L}_{\text{DPO}}(\theta) =$$

$$-\beta E_{(\bar{x},\bar{y}_w,\bar{y}_l)\sim\mathcal{D}}\left[\sigma\left(\hat{r}_\theta(\bar{x},\bar{y}_l) - \hat{r}_\theta(\bar{x},\bar{y}_w)\right)\left[\nabla \log \pi_\theta(\bar{y}_w \mid \bar{x}) - \nabla \log \pi_\theta(\bar{y}_l \mid \bar{x})\right]\right]$$

β functions like a "learning rate" following the strength of the KL constraint

Per-example weight: higher weight when the reward model is wrong

Increase likelihood of preferred example

Decrease likelihood of dispreferred example

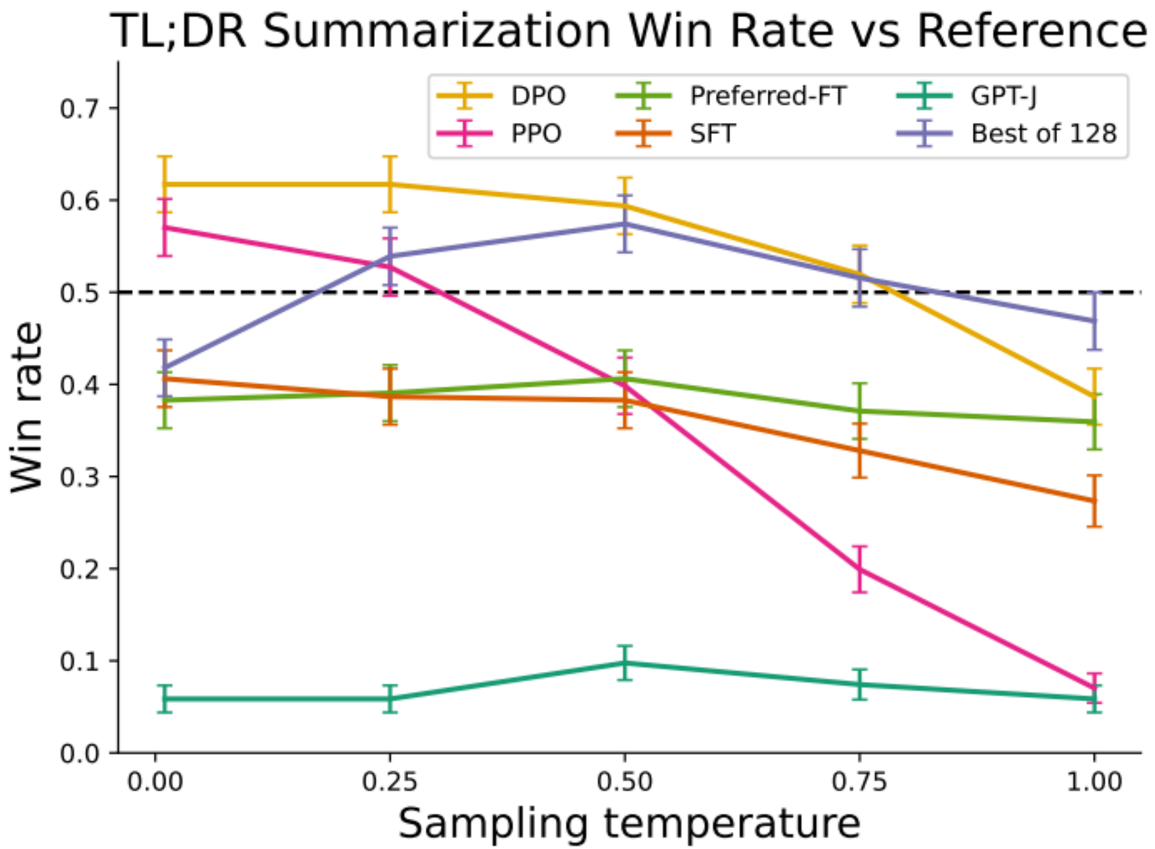
where $\hat{r}(\bar{x},\bar{y}) = \beta \log \frac{\pi_\theta(\bar{y} \mid \bar{x})}{\pi_{\text{ref}}(\bar{y} \mid \bar{x})}$

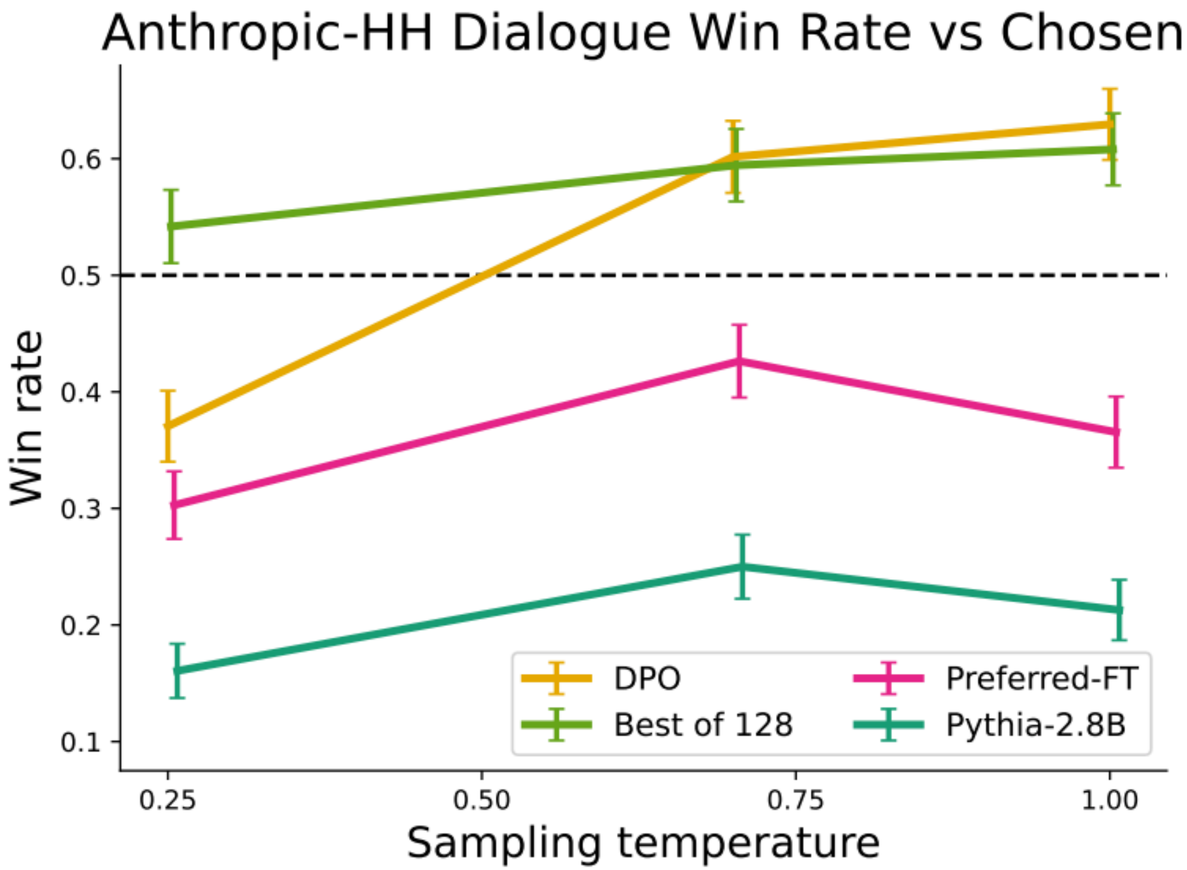
Slide credit: Yoav Artzi

Outcome of RLHF/DPO

- RLHF produces an “aligned” model that should achieve high reward
- Baselines:
  - Best-of-n: sample n responses from an SFT model, take the best one according to the reward function
    - Pro: training-free
    - Cons: expensive, may not deviate far from the initial SFT model
  - Preference tuning: apply SFT on preferred outputs
    - Pro: simple. Cons: doesn’t use the negative examples

DPO Results

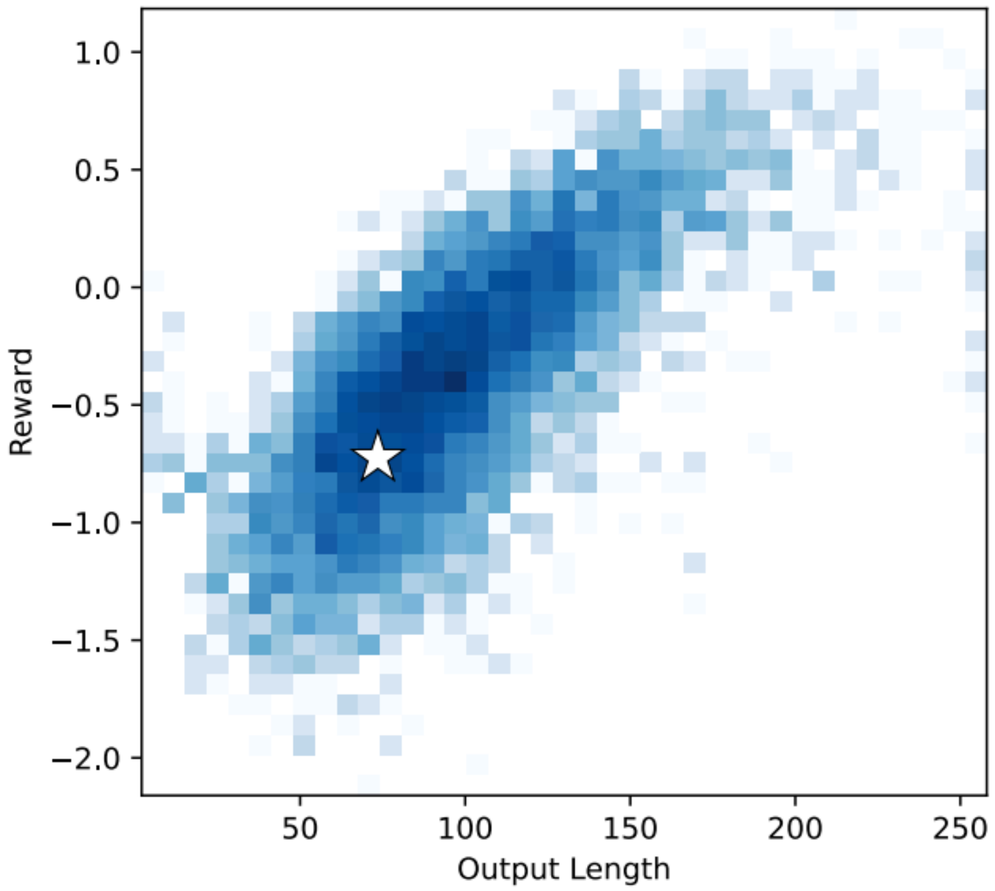




- Evaluation: *win rate* (as scored by an LLM)

Rafailov et al. (2023)

What does RLHF do?



Question: *Why don't adults roll off the bed?*

☆ **SFT (Before); 59 tokens**

Adults typically do not roll off of the bed because they have developed the muscle memory to keep their bodies from involuntarily moving during sleep and maintaining proper posture.

RLHF (After); 243 tokens: Similar output, but much longer / more details

Adults generally do not roll off of the bed because they have developed muscle control and awareness which helps them to remain upright while sleeping.

Additionally, most adults find it uncomfortable or even painful to move around in their sleep, so rolling off the bed would be difficult without waking up first. There may also be safety concerns such as falling out of bed and potentially hurting oneself if one were to lose balance or fall from the ...

- Reward models trained on open datasets have high correlations with length

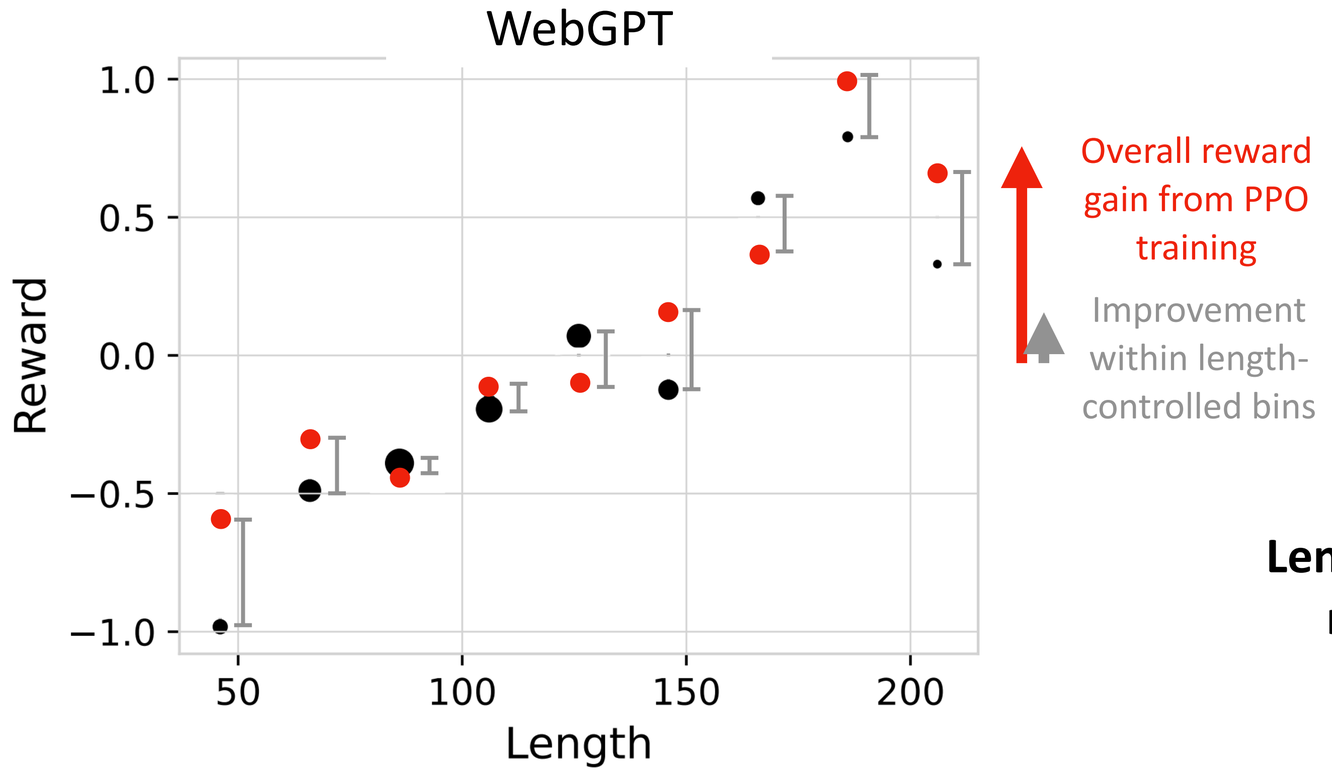
(Prasann Singhal, Tanya Goyal, Jiacheng Xu, GD, COLM 2024 oral spotlight)

What does RLHF do?

On older preference dataset, most reward optimization was attributable to shifting to longer outputs! (Modern datasets are much bigger and this effect is reduced)

Average bin reward (RLHF model outputs)

Average bin reward (SFT model outputs)



Overall reward gain from PPO training

Improvement within length-controlled bins



Length accounts for 85% of reward improvement

(Prasann Singhal, Tanya Goyal, Jiacheng Xu, GD, COLM 2024 oral spotlight)

Administrative details and recap

Supervised Fine-Tuning

SFT Data

RLHF

DPO

RLVR Intro

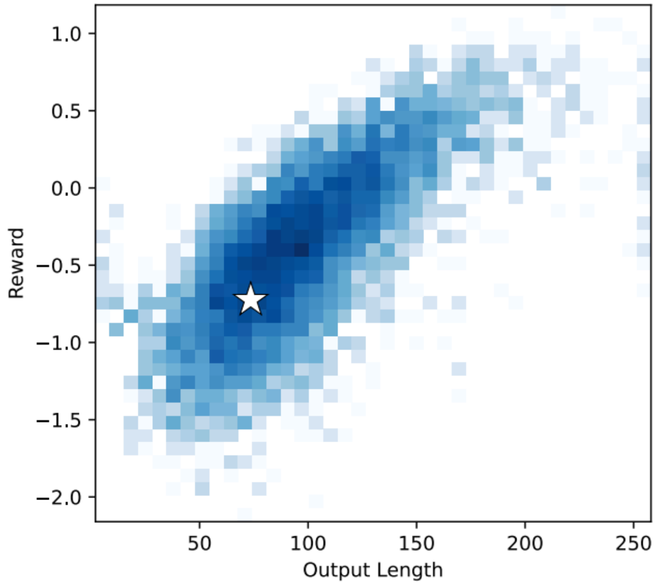
Basic RLVR: Expert Iteration,
Policy Gradient

GRPO

RLVR Intro

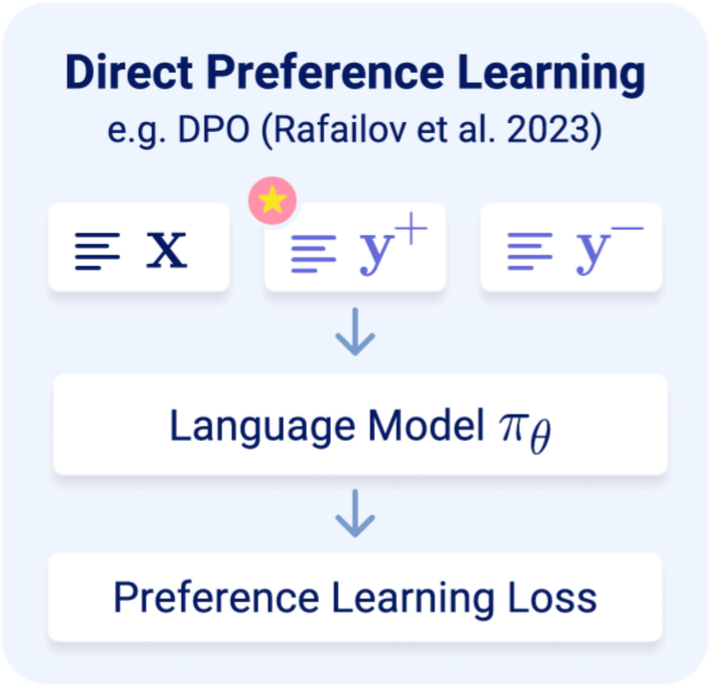
Shortcomings of RLHF

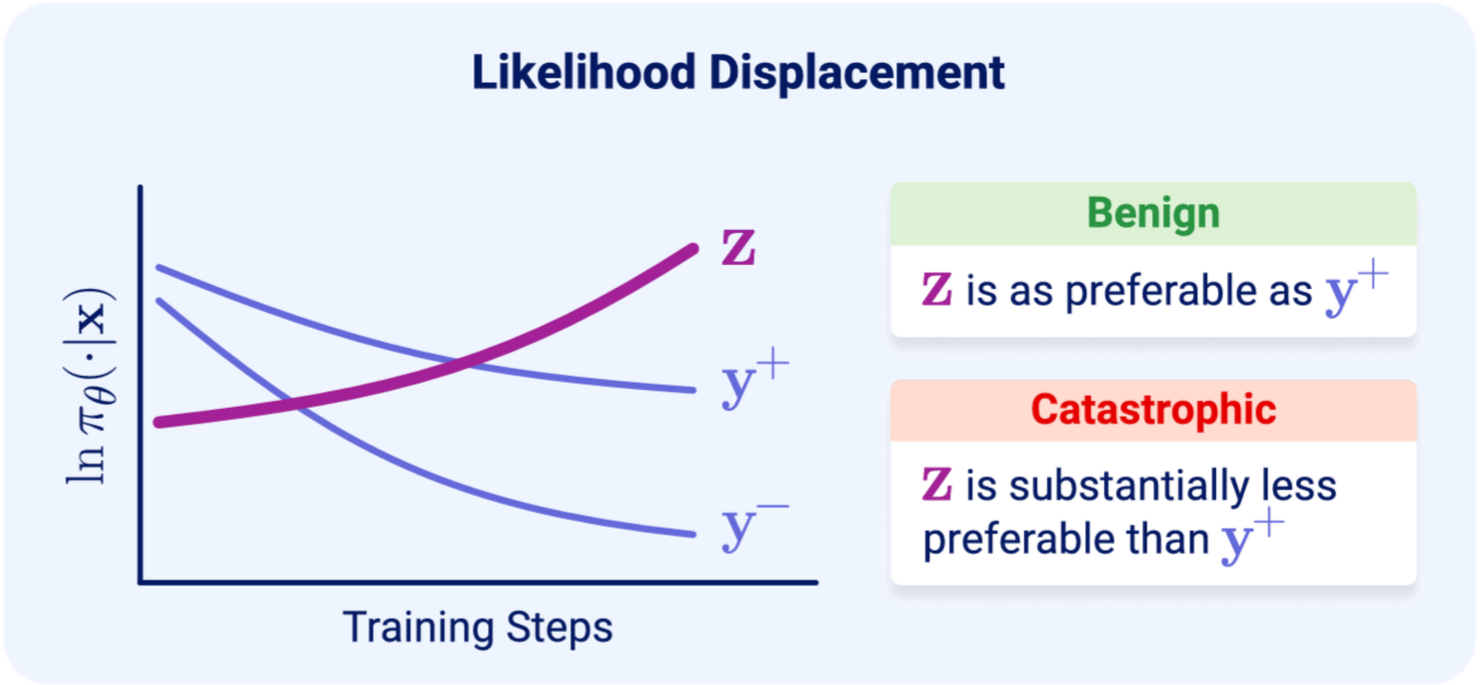
- Does RLHF “scale”? Can we do a very long (~pre-training length) RLHF run and get a very good LLM?
- Can’t run PPO indefinitely due to overoptimization



- DPO is mis-specified as an objective

Shortcomings of RLHF

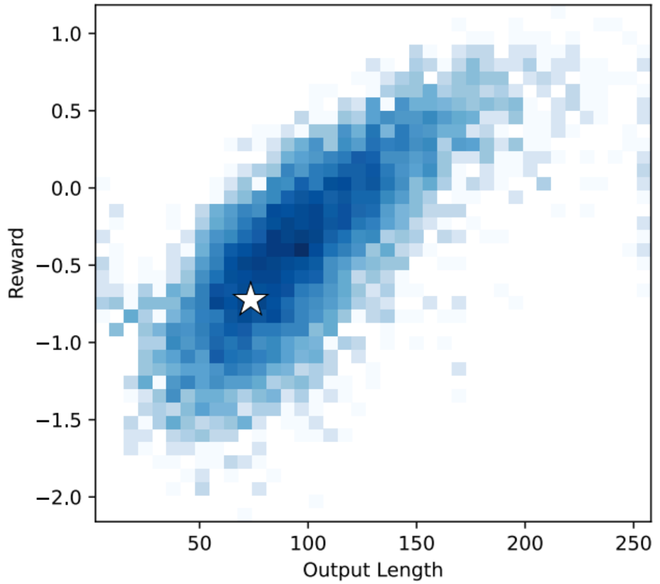




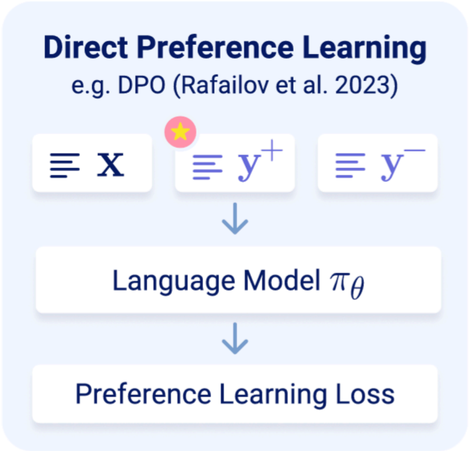
Noam Razin et al. (2025)

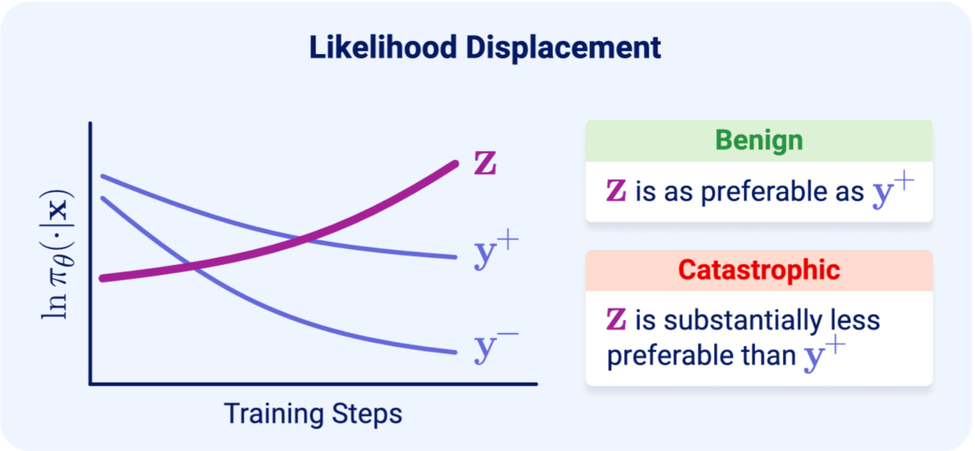
Shortcomings of RLHF

- Does RLHF “scale”? Can we do a very long (~pre-training length) RLHF run and get a very good LLM?
- Can’t run PPO indefinitely due to overoptimization



- DPO is mis-specified as an objective



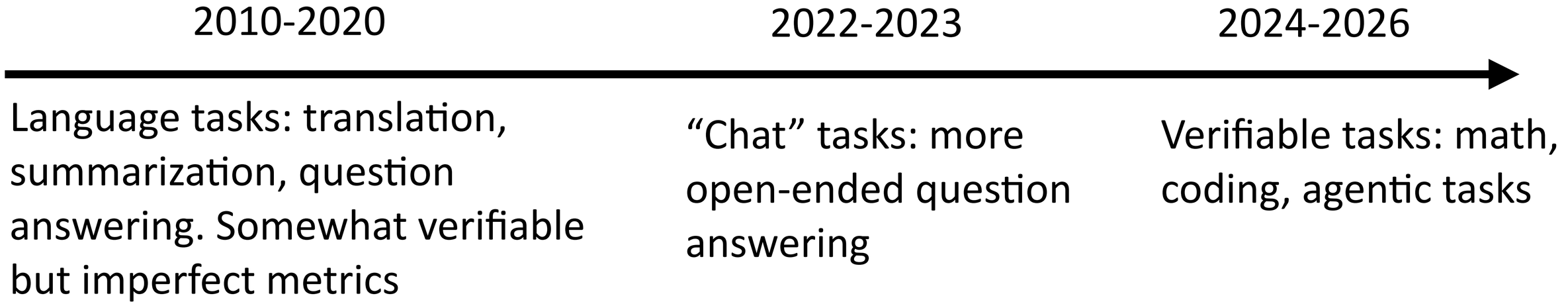


RLVR

“Reinforcement learning with verifiable rewards”

Verifiable reward: problem has a verifiable answer and we can say whether the model was correct or not

Why did RLVR “take over”? A shift in focus in where LLMs are being developed:



RLHF vs. RLVR

	RLHF	RLVR
Problems	“Chat” problems (“tell me three ways to live a healthy lifestyle)	Math, coding, agentic tasks (“make this repo compile + pass tests”)
Training data	(prompts x, preferred response y+, dispreferred response y-)	(prompts x, answer a)
Rewards	Model-based, “LLM-as-a-judge”	Verifiable: 1 or 0, correct/incorrect (small amount of recent work on LLM-as-a-judge)
Algorithms	PPO, DPO	PPO, GRPO

Answer Parsing

- Several formats for answers being output
- OpenThoughts:

  ***Final Answer**\nThe value of \\(k \\) is \\boxed{\\dfrac{-3 + \\sqrt{13}}{2}} or \ \boxed{\\dfrac{-3 - \\sqrt{13}}{2}}User: {question}*

- DeepSeek-R1 uses <answer> and </answer> to mark it
- In this assignment: you'll see how much answer parsing errors might be leading to issues
- For now: assume we have a function R(**y**, *a*) that can determine whether trace **y** output the answer *a*

Administrative details and recap

Supervised Fine-Tuning

SFT Data

RLHF

DPO

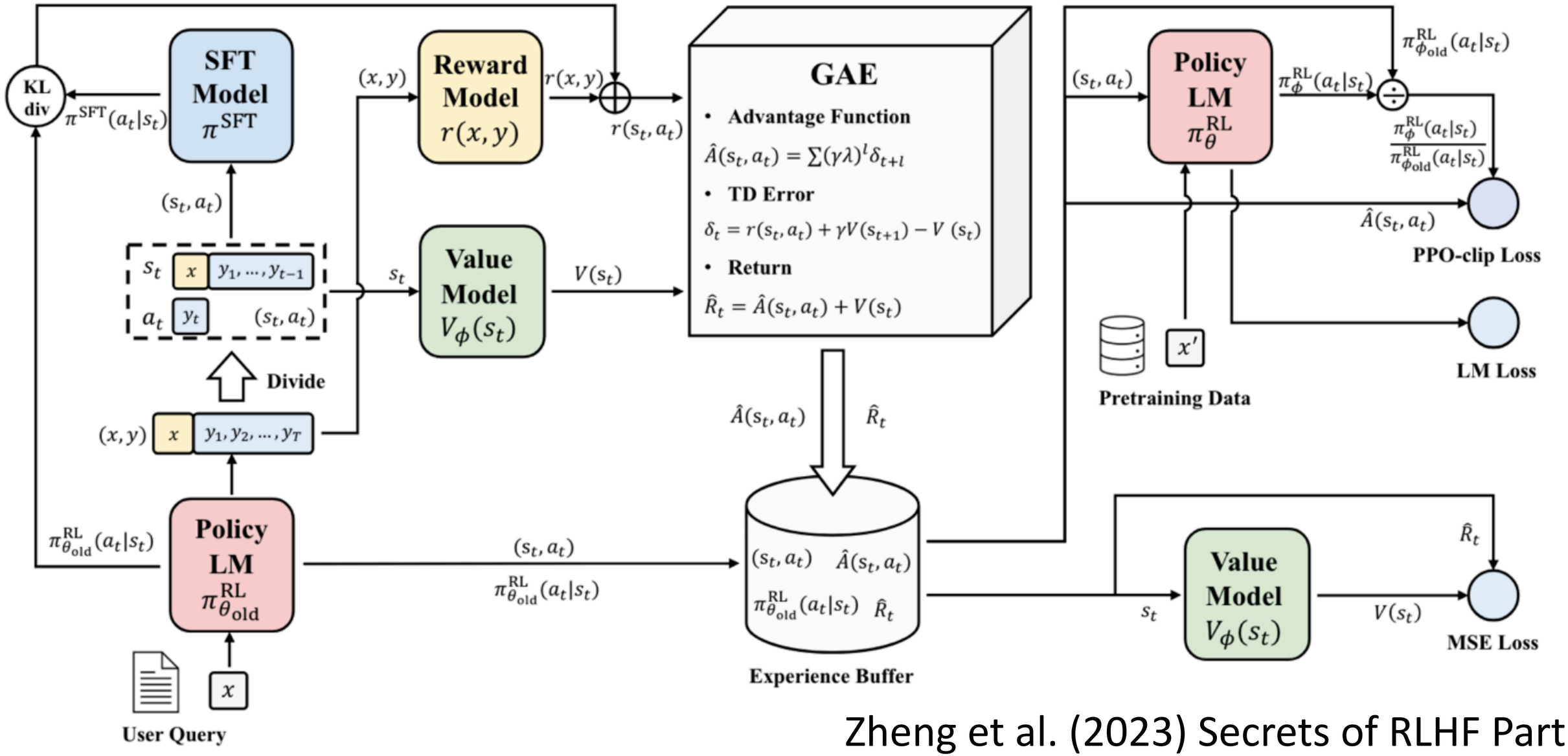
RLVR Intro

Basic RLVR: Expert Iteration,
Policy Gradient

GRPO

Basic RLVR: Expert Iteration, Policy Gradient

Shortcomings of PPO



Zheng et al. (2023) Secrets of RLHF Part 1

Simple Version: Expert Iteration

Examples $\{(\boldsymbol{x}_i, a_i)\}$ with inputs $\mathbf{x}$ and ground truth answers a

For each example $(\boldsymbol{x}_i , a_i)$

Do inference $\mathbf{y}_{pred} = \text{LLM}(\boldsymbol{x}_i)$

If $\mathbf{y}_{pred}$ is correct $(\text{R}(\mathbf{y}_{pred}, a_i) = 1)$

Train on $(\boldsymbol{x}_i, \mathbf{y}_{pred})$ with SFT

How does this differ from SFT?

Why is this better than SFT? Why could it be worse?

Zelikman et al. (2023) STaR: Bootstrapping Reasoning with Reasoning

Policy Gradient

Objective: $\sum_{\tau} P(\tau|\theta)R(\tau)$

Sample trajectories: run the language model to get output **y** | input **x**

View the output as a series of actions *a* (~tokens) given environment states (previous tokens emitted) *s*

Compute the return R (in our setting, reward)

Gradient step: $\mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)\big(R(\tau) - b(s_t)\big)\right]$

reward

SFT gradient on the trace

baseline (adjustment factor)

How does this differ from expert iteration?

Implementing Policy Gradient

$$\texttt{pg_loss} = \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T}\log \pi_\theta(a_t^{(i)}|s_t^{(i)})(R(\tau^{(i)}) - b(s_t^{(i)}))$$

When we implement policy gradient, we compute a “policy gradient loss”

Calling .backward() on this gives us the gradient, but *don’t* look at and monitor this loss! Instead monitor the reward value; we really care about optimizing that!

Administrative details and recap

Supervised Fine-Tuning

SFT Data

RLHF

DPO

RLVR Intro

Basic RLVR: Expert Iteration,
Policy Gradient

GRPO

GRPO

GRPO

sample problems q, then sample G rollouts o_i from the policy

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right)$$

remember policy gradient: reward * prob. This is that term.

Clipping: you "max out" if you've changed the ratio by at least epsilon

Ratio allows us to use "stale" rollouts when doing the update.

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}$$

group-normalized reward

GRPO: KL Penalty

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right)$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1$$

Modified form of KL with better numerical properties (never negative)

```python
# Process responses and calculate rewards
groups = [
    list(range(i, i + GENERATIONS_PER_SAMPLE)) for i in range(0, len(all_generations), GENERATIONS_PER_SAMPLE)
]  # example: [[0, 1, 2], [3, 4, 5], [6, 7, 8]]

all_query_token_ids, all_responses_token_ids, all_advantages = [], [], []

stats = {
    "response_lengths": [],
    "rewards": [],
    "non_stop_rate": [],
}

for sample, group_indices in zip(samples, groups):
    response_token_ids = [all_generations[i] for i in group_indices]
    finish_reasons = [all_finish_reasons[i] for i in group_indices]
    responses = tokenizer.batch_decode(response_token_ids, skip_special_tokens=False)
    rewards_and_metrics = [compute_reward(resp, sample, EOS_TOKEN) for resp in responses]
    rewards, reward_metrics = zip(*rewards_and_metrics)
```

Grab the samples: responses per each group

https://github.com/McGill-NLP/nano-aha-moment/

Group normalization

```
        rewards = np.array(rewards)
        advantages = (rewards - rewards.mean()) / (rewards.std() + 1e-4)

        per_token_advantages = [[adv] * len(resp) for adv, resp in zip(advantages, response_token_ids)]

        all_query_token_ids.extend([sample["input_ids"]] * GENERATIONS_PER_SAMPLE)
        all_responses_token_ids.extend(response_token_ids)
        all_advantages.extend(per_token_advantages)

        stats["rewards"].extend(rewards)
        stats["non_stop_rate"].extend([fr != "stop" for fr in finish_reasons])
        stats["response_lengths"].extend([len(ids) for ids in response_token_ids])
        for rm in reward_metrics:
            for k, v in rm.items():
                stats.setdefault(f"reward_metrics/{k}", []).append(v)

episodes = {
    "all_query_token_ids": all_query_token_ids,
    "all_response_token_ids": all_responses_token_ids,
    "all_advantages": all_advantages,
}
```

```python
def compute_pg_loss(
    policy_model: Union[DeepSpeedEngine, PreTrainedModel],
    batch: Dict[str, torch.Tensor],
    total_response_len: torch.Tensor,
    TEMPERATURE: float,
    KL_COEFFICIENT: float,
) -> Tuple[torch.Tensor, Dict[str, float]]:
    """
    Compute the policy gradient loss with KL penalty between policy and reference models.

    This function:
    1. Calculates KL divergence penalty between the models
    2. Computes policy gradient loss using advantages
    3. Combines the losses with KL coefficient

    Args: [I cut here]
    Returns:
        Tuple containing:
            - loss: Combined policy gradient and KL penalty loss (scalar tensor)
            - metrics: Dictionary with detailed loss components:
                - policy_loss: Pure policy gradient loss
                - kl_penalty: KL divergence penalty
                - entropy: Policy entropy
    """
```

```
logps = compute_token_log_probs(policy_model, model_inputs, TEMPERATURE)  # [batch_size, seq_len-1]
labels_mask = labels_mask[..., 1:].to(logps.dtype)  # [batch_size, seq_len-1]

ref_logratio = ref_logps - logps
kl_penalty = torch.exp(ref_logratio) - 1 - ref_logratio  # [batch_size, seq_len-1]
kl_penalty = kl_penalty * labels_mask  # [batch_size, seq_len-1]

with torch.no_grad():
    entropy = -logps.sum() / labels_mask.sum()  # scalar
    zero_advantages = close_to_zero(advantages[..., 1:], labels_mask)  # scalar

policy_loss = -logps * advantages[..., 1:]  # [batch_size, seq_len-1]
policy_loss = policy_loss * labels_mask  # [batch_size, seq_len-1]

loss = (policy_loss + KL_COEFFICIENT * kl_penalty).sum() / total_response_len  # scalar

metrics = {
    "policy_loss": policy_loss.sum().item() / total_response_len.item(),
    "kl_penalty": kl_penalty.sum().item() / total_response_len.item(),
    "entropy": entropy.item() / total_response_len.item(),
    "zero_advantages_ratio": zero_advantages.item() / total_response_len.item(),
}
```

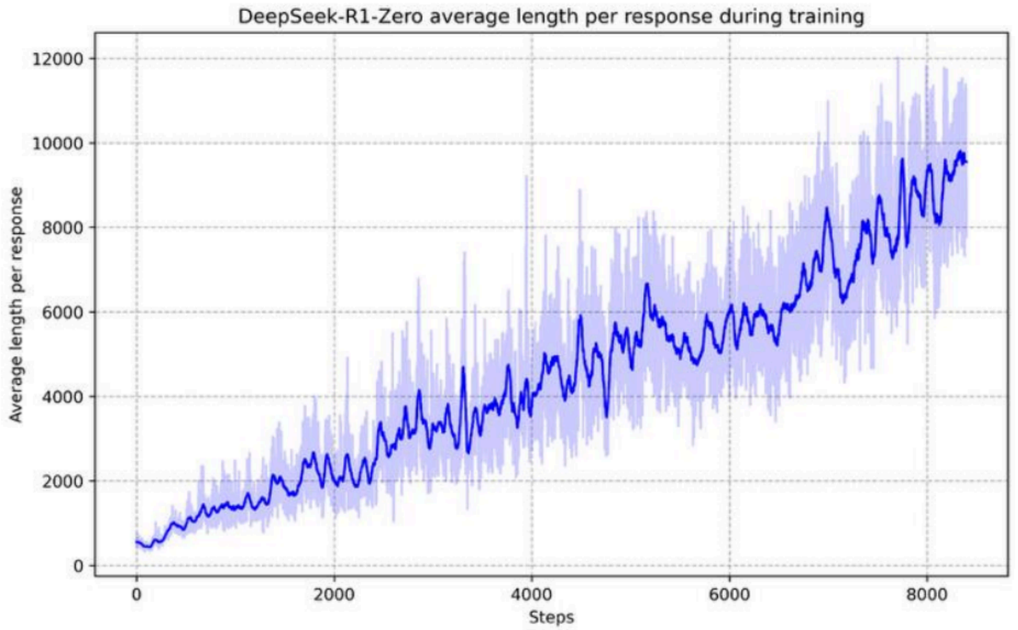
What is this part doing?

What is this part doing?

Advantages of GRPO

- Group rollout
- Compute advantage relative to the group (z-score within the group)
- No value model!

Result: “Long Chains of Thought”



Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$

$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$

Rearrange to isolate the inner square root term:

$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$

$\ldots$

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$

We started with the equation:

$\sqrt{a - \sqrt{a + x}} = x$

First, let's square both sides:

$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$

Next, I could square both sides again, treating the equation: $\cdots$

$\ldots$

Variants of GRPO

- There are a million of them! One example: Dr. GRPO:

GRPO

$$\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|\mathbf{o}_i|}\sum_{t=1}^{|\mathbf{o}_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}\hat{A}_{i,t},\text{clip}\left(\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]\right\},$$

$$\text{where } \hat{A}_{i,t}=\frac{R(\mathbf{q},\mathbf{o}_i)-\text{mean}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})}{\text{std}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\})}.$$

Dr. GRPO

GRPO Done Right (without bias)

$$\frac{1}{G}\sum_{i=1}^{G}\sum_{t=1}^{|\mathbf{o}_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}\hat{A}_{i,t},\text{clip}\left(\frac{\pi_\theta(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|\mathbf{q},\mathbf{o}_{i,<t})},1-\epsilon,1+\epsilon\right)\hat{A}_{i,t}\right]\right\},$$

$$\text{where } \hat{A}_{i,t}=R(\mathbf{q},\mathbf{o}_i)-\text{mean}(\{R(\mathbf{q},\mathbf{o}_1),\ldots,R(\mathbf{q},\mathbf{o}_G)\}).$$

DrGRPO

High stddev, low updates

Low stddev, updates amplified

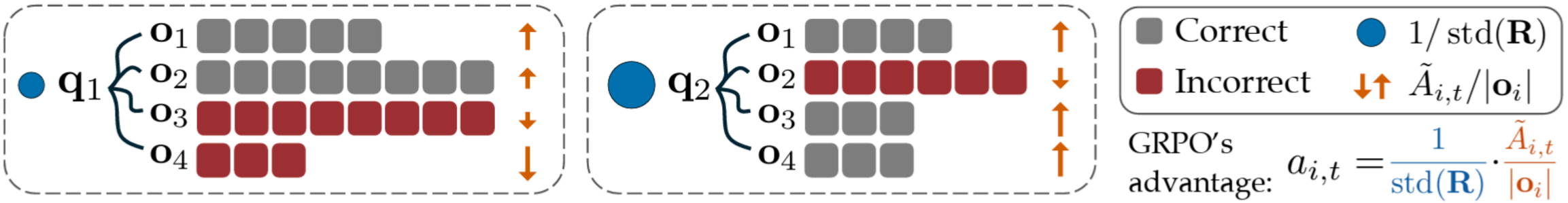
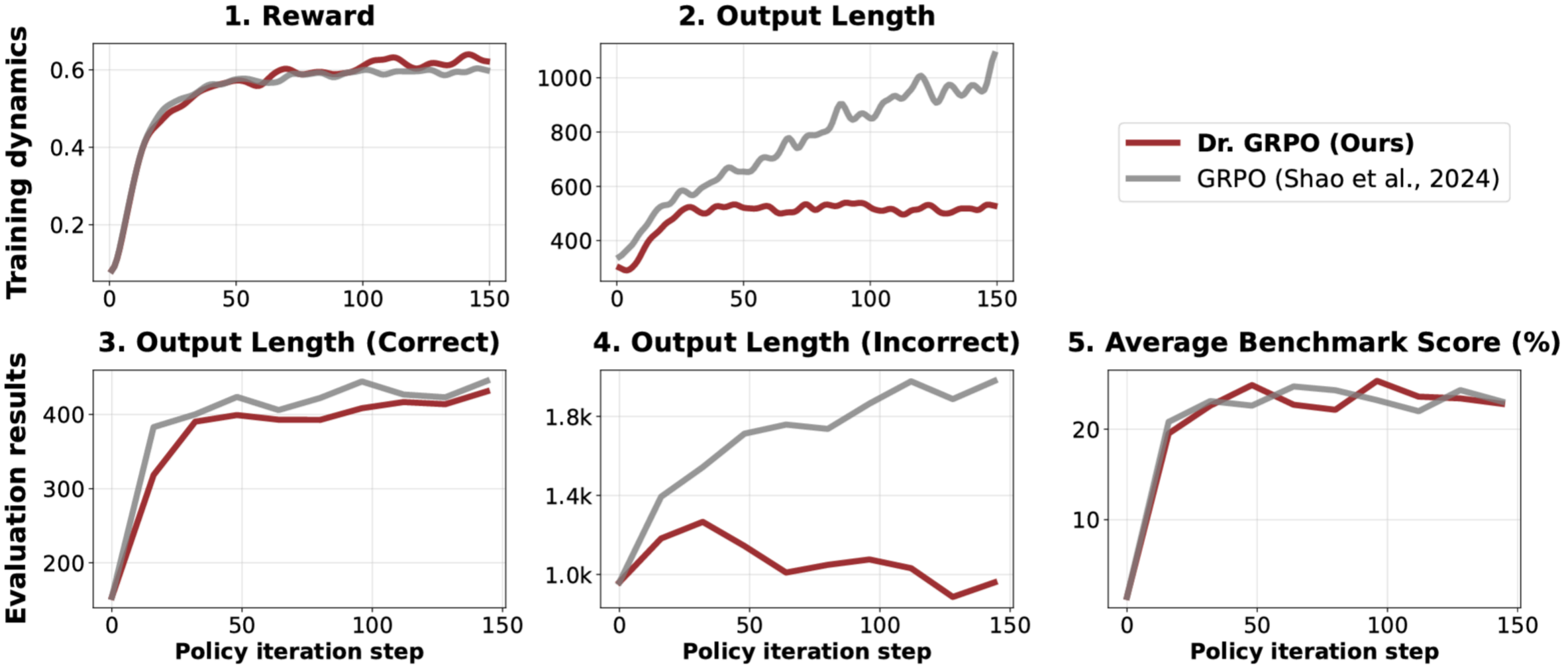


Figure 4: Illustration of the biases in GRPO. Note that the effective advantage of GRPO $a_{i,t}$ is equivalent to a reweighted version of the unbiased advantage $\tilde{A}_{i,t} = R(\mathbf{q}, \mathbf{o}_i) - \mathrm{mean}(\mathbf{R})$. The terms $\mathrm{std}(\mathbf{R})$ and $|\mathbf{o}_i|$ could bias the optimization by assigning different weights to different questions and responses, as denoted by the sizes of the blue circles and the lengths of the orange arrows. Upward arrows indicate positive advantages, and vice versa.

Shorter correct responses are boosted more, longer incorrect responses are penalized less

Also excludes KL term

DrGRPO


1. Reward
2. Output Length
3. Output Length (Correct)
4. Output Length (Incorrect)
5. Average Benchmark Score (%)
Training dynamics
Evaluation results
Dr. GRPO (Ours)
GRPO (Shao et al., 2024)
Policy iteration step
Policy iteration step
Policy iteration step

Administrative details and recap

Supervised Fine-Tuning

SFT History

Understanding SFT Data

RLHF

DPO

Putting it together: Olmo3, OpenThoughts